데이터사이언스

Lecture 11 데이터 프레임과 통계 함수

기본통계량 – describe()

```
# ── 수치형 열 기술통계 ──────────────────────────────
df.describe()
# count   mean   std   min   25%   50%   75%   max

# ── 범주형 열 기술통계 (include="object") ──────────
df.describe(include="object")
# count   unique   top   freq

# ── 모든 열 한번에 ──────────────────────────────────
df.describe(include="all")

# ── 퍼센타일 직접 지정 ──────────────────────────────
df.describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9])

# ── 수치형만 / 범주형만 따로 ────────────────────────
df.select_dtypes(include="number").describe()
df.select_dtypes(include="object").describe()
```

count ≠ len(df)

count < len(df)이면
결측치가 있다는 신호

mean vs 50%

둘이 크게 다르면
이상치 또는 왜도 존재

min/max 확인

비현실적 값이면
이상치 의심

std 크기

mean 대비 std가 크면
데이터가 넓게 퍼져있음

범주형 열 탐색 – value_counts() / nunique() / unique()

```
# ── value_counts() : 값별 빈도 ──────────────────────────────
df["등급"].value_counts()           # 빈도 내림차순
df["등급"].value_counts(normalize=True)  # 비율로 출력 (합계=1)
df["등급"].value_counts(dropna=False)    # NaN도 포함해서 세기

# ── nunique() : 고유값 개수 ─────────────────────────────────
df["등급"].nunique()                # 고유 범주 수
df.nunique()                        # 전체 열의 고유값 수 한번에

# ── unique() : 고유값 목록 ──────────────────────────────────
df["등급"].unique()                 # 어떤 값들이 있는지 확인
df["등급"].nunique() == 2           # 이진 범주인지 확인

# ── 활용: 전처리 필요 여부 빠른 체크 ────────────────────────
for col in df.select_dtypes("object").columns:
    print(f"{col}: {df[col].nunique()}개 → {df[col].unique()[:5]}")
```

분포의 모양 – skew() / kurt()

```
# 왜도 / 첨도 계산
df["나이"].skew()          # 왜도
df["나이"].kurt()          # 첨도 (pandas: excess kurtosis)

# 전체 수치형 열 한번에
df.select_dtypes("number").skew().sort_values(ascending=False)
df.select_dtypes("number").kurt().sort_values(ascending=False)

# 왜도 높은 열 → 로그 변환 필요 여부 판단
import numpy as np
skewed_cols = df.select_dtypes("number").skew()
skewed_cols = skewed_cols[abs(skewed_cols) > 1.0].index.tolist()
print("로그변환 고려 열:", skewed_cols)

# 로그 변환 후 왜도 비교
df["소득_log"] = np.log1p(df["소득"])
print(f"변환 전: {df['소득'].skew():.2f}")
print(f"변환 후: {df['소득_log'].skew():.2f}")
```

왜도 (Skewness)

분포의 좌우 비대칭 정도
- 0 = 대칭
- 양수 = 오른쪽 꼬리
- 음수 = 왼쪽 꼬리

첨도 (Kurtosis)

분포의 뾰족함 정도
- 0 = 정규분포
- 양수 = 뾰족
- 음수 = 납작

기술통계 예제

"week11_shop.csv" 는 온라인 쇼핑몰 고객의 구매 데이터이다. 변수 설명은 다음과 같다.
(encoding = "cp949"로 불러올 것)

변수	설명
고객ID	고객 고유 번호
나이	고객 나이 (세)
연소득	연간 소득 (만원)
구매금액	총 구매금액 (원)
구매횟수	총 구매 횟수
성별	남 / 여
등급	BRONZE / SILVER / GOLD / VIP
지역	서울 / 경기 / 부산

1) 수치형 열의 기술통계를 확인해보자.

2) 범주형 열의 기술통계를 확인해보자. 등급 열에서 가장 자주 등장하는 값은?

3) 수치형 열의 왜도가 1이 초과하는 변수는? 이유는?

상관계수 행렬 – corr()

범위	해석	의미
+0.7 ~ +1.0	강한 양의 상관	X↑ → Y↑
+0.3 ~ +0.7	중간 양의 상관	약하게 같이 움직임
-0.3 ~ +0.3	거의 무상관	관계 없음
-0.7 ~ -0.3	중간 음의 상관	약하게 반대로 움직임
~1.0 ~ -0.7	강한 음의 상관	X↑ → Y↓

```
# 기본 상관계수 행렬 (Pearson)
df.corr()
df.corr(method="pearson")   # 기본값
df.corr(method="spearman")  # 순위 기반 (이상치에 강함)
df.corr(method="kendall")   # 순서 일치 비율

# 특정 열과의 상관만 보기
df.corr()["목표변수"].sort_values(ascending=False)

# 강한 상관 쌍만 뽑기
corr_mat = df.corr().abs()
upper = corr_mat.where(
    np.triu(np.ones(corr_mat.shape), k=1).astype(bool)
)
strong_pairs = [(col, row, corr_mat.loc[row,col])
    for col in upper.columns
    for row in upper.index
    if upper.loc[row,col] > 0.7]
print(strong_pairs)
```

상관관계 해석 주의사항

- 상관 ≠ 인과: 아이스크림 판매량과 익사 사고는 높은 상관관계. 원인은 '여름'이라는 숨겨진 변수.
- 다중공선성: 두 독립변수가 서로 강하게 상관될 때 발생. 모델이 각 변수의 효과를 분리 못함.
- 해결방법: 상관 0.8 이상 변수 중 하나 제거. PCA로 차원 축소. VIF 계산.

```
# 다중공선성 확인 – 상관 0.8 이상 쌍 찾기
corr_mat = df.select_dtypes("number").corr().abs()
high_corr = corr_mat[(corr_mat > 0.8) & (corr_mat < 1.0)]
print(high_corr.stack().dropna().reset_index())

# VIF (Variance Inflation Factor) 계산
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = df[["나이","소득","경력"]].dropna()
vif_df = pd.DataFrame({
    "Feature": X.columns,
    "VIF": [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})
print(vif_df)
# VIF > 10 → 심각한 다중공선성 → 해당 변수 제거 고려, VIF > 5  → 주의 필요
```

상관계수 예제

"week11_shop.csv" 를 이용하여 다음 문제를 푸세요.

1) 구매금액과 가장 강한 상관관계를 가진 열은?
2) 나이와 연소득의 상관계수는?
3) 앞에서 나이와 연소득에 이상치가 있는 것을 확인하였다. 이상치를 평균으로 대체하고 나이와 연소득의 상관계수를 구해보자.
4) 상관계수가 0.8이상 1.0이하로 높은 상관관계를 가진 쌍은?
5) 나이, 연소득, 구매금액의 VIF는 어떻게 되는가?

그룹 통계 – transform()

- agg(): 그룹별 요약하여 행 수가 줄어듦. 그룹당 하나의 값. 상권A → 평균이익 값 하나
- transform(): 원래 행 수 유지. 각 행에 그룹 통계 붙임. 상권A의 모든 행 → 같은 평균이익 값

```
# ── transform() 핵심 패턴 ──────────────────────────────────
# 그룹 평균을 각 행에 붙이기 (행 수 유지!)
df["상권_평균이익"] = df.groupby("상권")["이익"].transform("mean")

# 그룹 내 표준화 (z-score)
df["이익_표준화"] = df.groupby("상권")["이익"].transform(lambda x: (x - x.mean()) / x.std())

# 그룹 내 순위
df["상권내_순위"] = df.groupby("상권")["이익"].transform(lambda x: x.rank(ascending=False))

# 그룹 최댓값 대비 비율
df["최대대비비율"] = df["이익"] / df.groupby("상권")["이익"].transform("max")

# 결측치를 그룹 평균으로 대체
df["이익"] = df["이익"].fillna(df.groupby("상권")["이익"].transform("mean"))
```

그룹 통계 – filter() / agg()

```
# ── filter() – 그룹 조건으로 행 필터링 ──────────────────────
# 평균 이익이 500000 이상인 상권의 행만 유지
df_filtered = df.groupby("상권").filter(lambda x: x["이익"].mean() >= 500000)

# 행이 5개 이상인 그룹만 유지
df_filtered = df.groupby("상권").filter(lambda x: len(x) >= 5)

# 이익이 모두 양수인 그룹만 유지
df_filtered = df.groupby("상권").filter(lambda x: (x["이익"] > 0).all())

# ── agg() 커스텀 함수 ──────────────────────────────────────
# 여러 집계 동시에 + 커스텀 함수
df.groupby("상권")["이익"].agg(["mean", "std", "count",
    ("범위", lambda x: x.max() - x.min()),          # 범위
    ("상위25%", lambda x: x.quantile(0.75)),         # 3사분위수
    ("이익률", lambda x: (x > 0).sum() / len(x)),   # 흑자 비율])

# 열마다 다른 집계
df.groupby("상권").agg({"이익":  ["mean","std"], "매출":  ["sum","max"], "고객수": "mean"})
```

그룹 통계 – apply() + groupby

```
# ── groupby + apply ──────────────────────────────────────────
# 각 상권의 상위 3개 매장만 뽑기
top3_per_group = df.groupby("상권").apply(lambda x: x.nlargest(3, "이익")).reset_index(drop=True)

# 각 그룹 내에서 이상치 제거 (IQR)
def remove_outliers(group):
    Q1 = group["이익"].quantile(0.25)
    Q3 = group["이익"].quantile(0.75)
    IQR = Q3 - Q1
    return group[(group["이익"] >= Q1-1.5*IQR) &
                 (group["이익"] <= Q3+1.5*IQR)]

df_clean = df.groupby("상권").apply(remove_outliers).reset_index(drop=True)

# 그룹별 상관계수 계산
corr_by_group = df.groupby("상권").apply(
    lambda x: x["매출"].corr(x["이익"])
).rename("매출_이익_상관")
print(corr_by_group)
```

그룹 통계 예제

"week11_shop.csv" 를 이용하여 다음 문제를 푸세요.

1) transform()을 사용해서 등급별 평균 구매금액을 각 행에 붙인 뒤, SILVER 등급의 등급별_평균구매 값은?
2) 평균 구매금액이 300,000원 이상인 등급의 행만 필터링해보자.

분포 파악 & 이상치 통계 – quantile() & IQR

```
# ── quantile() 활용 ──────────────────────────────────────
df["이익"].quantile(0.25)                    # Q1
df["이익"].quantile([0.1, 0.25, 0.5, 0.75, 0.9])  # 여러 분위수

# ── IQR 이상치 자동 탐지 함수 만들기 ──────────────────────
def detect_outliers_iqr(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(f"[{col}] Q1={Q1:.1f}, Q3={Q3:.1f}, IQR={IQR:.1f}")
    print(f"  하한={lower:.1f}, 상한={upper:.1f}, 이상치={len(outliers)}개")
    return outliers

# 모든 수치형 열에 한번에 적용
for col in df.select_dtypes("number").columns:
    detect_outliers_iqr(df, col)
```

분포 파악 & 이상치 통계 – quantile() & IQR

```
# ── 이상치 처리 전략 선택 ──────────────────────────────────
# 전략1: 제거
df_clean = df[(df["이익"] >= lower) & (df["이익"] <= upper)]

# 전략2: 경계값으로 클리핑
df["이익"] = df["이익"].clip(lower=lower, upper=upper)

# 전략3: 이상치를 NaN으로 → fillna로 대체
df.loc[(df["이익"] < lower)|(df["이익"] > upper), "이익"] = np.nan
df["이익"] = df["이익"].fillna(df["이익"].median())
```

분포 파악 예제

"week11_shop.csv" 를 이용하여 다음 문제를 푸세요.

1) IQR 방법으로 연소득 열의 이상치를 탐지했을 때, 이상치 수는?
2) 모든 수치형 열에 대해 이상치를 확인하는 detect_outliers_iqr 함수를 만들고 각 열의 이상치 개수를 확인해보자.
3) "나이" 열에서 이상치를 제거하지 않고 경계값으로 대체하려고 한다. 코드가 어떻게 되는가?

교차분석 – crosstab()

```
# ── 기본 교차표 ──────────────────────────────────────────
pd.crosstab(df["성별"], df["등급"])

# ── normalize: 비율로 변환 ──────────────────────────────
# 전체 대비 비율
pd.crosstab(df["성별"], df["등급"], normalize=True)

# 행 기준 비율 (각 성별 내 등급 비율)
pd.crosstab(df["성별"], df["등급"], normalize="index")

# 열 기준 비율 (각 등급 내 성별 비율)
pd.crosstab(df["성별"], df["등급"], normalize="columns")

# ── margins: 합계 추가 ──────────────────────────────────
pd.crosstab(df["성별"], df["등급"], margins=True, margins_name="합계")

# ── 값 집계 추가 ────────────────────────────────────────
pd.crosstab(df["성별"], df["등급"], values=df["이익"], aggfunc="mean")
```

다중 집계 – pivot_table()

```
# ── 기본 구조 ──────────────────────────────────────────
pd.pivot_table(df, values="이익", index="상권", columns="창업가능", aggfunc="mean", fill_value=0)

# ── 여러 집계 함수 동시에 ──────────────────────────────
pd.pivot_table(df, values="이익", index="상권",
               aggfunc=["mean","sum","count", ("흑자비율", lambda x: (x>0).mean())])

# ── 여러 값 열 동시에 ──────────────────────────────────
pd.pivot_table(df,
               values=["이익","매출"], index="상권", columns="창업가능",
               aggfunc={"이익":"mean", "매출":"sum"}, fill_value=0)

# ── margins + 비율 계산 ────────────────────────────────
pt = pd.pivot_table(df, values="이익", index="상권",
                    aggfunc="sum", margins=True, margins_name="전체합계")

# 전체 대비 비율 열 추가
pt["비율(%)"] = (pt["이익"] / pt.loc["전체합계","이익"] * 100).round(1)
print(pt)
```

교차 분석 및 다중 집계 예제

"week11_shop.csv" 를 이용하여 다음 문제를 푸세요.

1) 성별과 등급의 교차표를 만들었을 때, 남성 중 GOLD 등급 고객 수는?
2) 성별 내 등급 비율을 구해보자.

EDA 체크리스트

01 기본 구조

shape / dtypes / head() / info()

02 결측치 현황

isnull().sum() → 비율 확인 → 처리 전략

03 수치형 분포

describe() / skew() / kurt() → 왜도 높으면 로그변환

04 이상치 탐지

IQR 또는 Z-Score → 제거/클리핑/대체

05 범주형 탐색

value_counts() / nunique() → 인코딩 전략 결정

06 상관관계

corr() → 목표변수와 강한 상관 / 다중공선성 탐지

07 그룹 비교

groupby + 통계 → 그룹별 패턴 발견

08 교차분석

crosstab() → 범주형 × 범주형 관계

EDA 루틴 — Step 1 & 2

```
df = pd.read_csv("data.csv")

# == Step 1: 기본 구조 파악 ==========================================
print(f"크기: {df.shape[0]}행 × {df.shape[1]}열")
print(df.dtypes)
print(df.head())

# == Step 2: 결측치 & 타입 점검 ======================================
missing = df.isnull().sum()
missing_rate = missing / len(df) * 100
print(pd.DataFrame({"결측수": missing, "결측률(%)": missing_rate.round(1)})
      [missing > 0].sort_values("결측률(%)", ascending=False))

print("\n=== 타입 확인 ===")
print(df.info())

print("\n=== 고유값 수 (범주형 열) ===")
for col in df.select_dtypes("object").columns:
    print(f"  {col}: {df[col].nunique()}개 → {df[col].unique()[:3]}")
```

EDA 루틴 — Step 3

```
# ══ Step 3: 분포 & 이상치 탐지 ═══════════════════════════
print("=== 기술통계 ===")
print(df.describe().T)  # .T 로 전치 → 열이 많을 때 보기 편함

print("\n=== 왜도 (|왜도| > 1 주의) ===")
skew = df.select_dtypes("number").skew().sort_values(ascending=False)
print(skew[abs(skew) > 0.5])

print("\n=== 이상치 탐지 (IQR) ===")
for col in df.select_dtypes("number").columns:
    Q1, Q3 = df[col].quantile([0.25, 0.75])
    IQR = Q3 - Q1
    n_out = ((df[col] < Q1-1.5*IQR) | (df[col] > Q3+1.5*IQR)).sum()
    if n_out > 0:
        print(f"  {col}: {n_out}개 이상치")
```

EDA 루틴 — Step 4

```
# ═══ Step 4: 관계 & 그룹 분석 ═══════════════════════════════
print("\n=== 상관관계 (목표변수 기준) ===")
target = "이익"
print(df.corr()[target].sort_values(ascending=False))

print("\n=== 범주형 × 수치형 그룹 비교 ===")
for col in df.select_dtypes("object").columns:
    print(f"\n[{col}별 {target} 평균]")
    print(df.groupby(col)[target].mean().sort_values(ascending=False))
```

11주차 과제

"week11_item.csv" 를 이용하여 eclass 내 과제를 푸세요.
(encoding="utf-8-sig"로 불러올 것)

변수	설명
상품ID	상품 고유 번호
카테고리	전자기기 / 패션 / 식품
가격	상품 가격 (원)
평점	고객 평균 평점 (0~5)
리뷰수	누적 리뷰 수
재고	현재 재고 수량
할인여부	Y / N